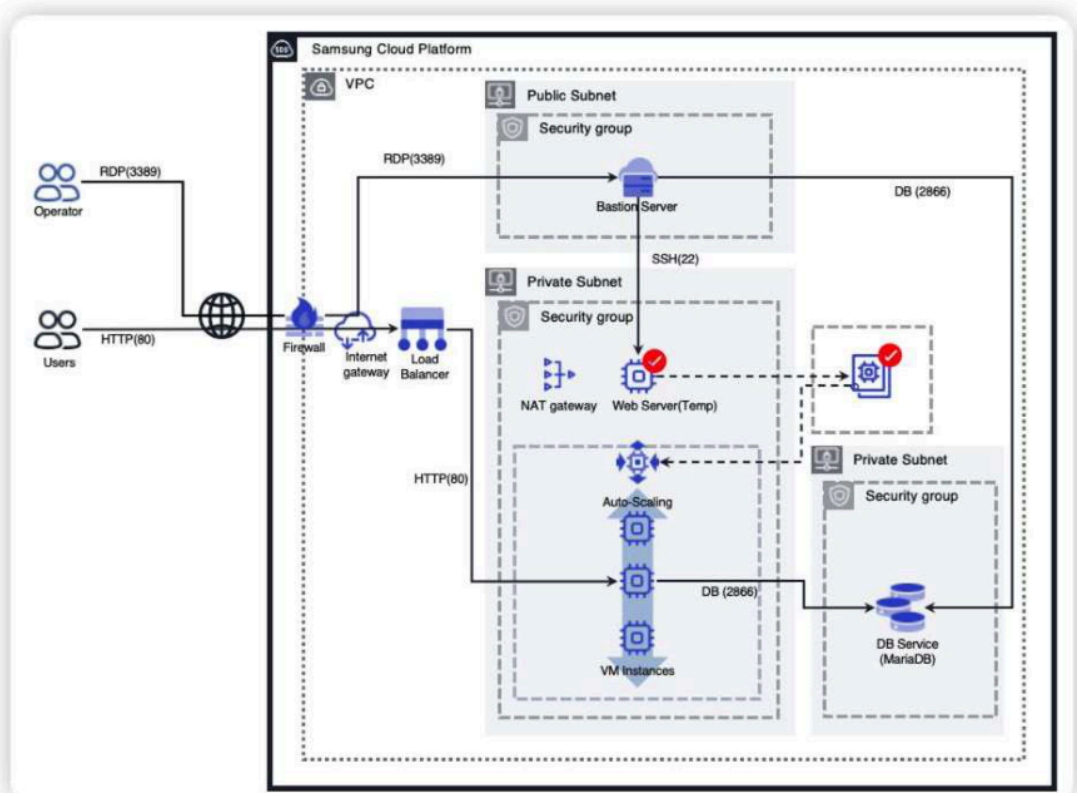
1. 전체 아키텍처 - 무엇을 왜 이렇게 배치했나



Hands-on Lab I(IaaS/DBaaS)이 단계별로 완성해 가는 구성. 사용자 경로와 운영자 경로가 처음부터 분리되어 있음

구성 요소	배치	설계 의도	막으려는 문제
IGW + Firewall	VPC 경계	인터넷 출입구를 한 곳으로 모음	통제되지 않은 출입 경로
Load Balancer	VPC 경계 안쪽	사용자가 닿는 곳은 LB 하나	서버 직접 노출, 서버 1대 장애 = 서비스 장애
Bastion Server	Public Subnet	관리 접속 통로를 한 곳으로	관리 포트(22·3389) 인터넷 노출
Web(Auto-Scaling)	Private Subnet	공인 IP 없이 LB 뒤에서만 동작	서버 단위 공격, 부하 급증 시 장애
NAT Gateway	Private Subnet	내부에서 나가는 통신만 허용	패치 불가 또는 서버별 공인 IP 남발
DB Service	별도 Private Subnet	Web·Bastion에서 DB 포트로만 접근	데이터 계층 직접 노출, 측면 이동
Custom Image	이미지 저장소	한 번 구성한 서버를 복제	서버마다 다른 설정, 빈 서버 Scale-Out

이 구성에 깔린 설계 원칙

- 최소 노출: 인터넷에서 닿는 곳은 LB(서비스)와 Bastion(관리) 두 곳뿐
- 다층 방어: Firewall(경계) → Security Group(자원) → Subnet 위치(Public/Private)
- 장애 전제: LB 헬스체크, Auto-Scaling, DB Active-Standby로 "한 대는 언제든 죽는다"를 가정
- 각 장은 이 원칙이 구성도의 어느 부분에서 구현되는지, 빠지면 무슨 일이 생기는지를 다룸

2. 서브넷을 역할별로 나누는 이유

구성: 하나의 VPC 안에 Public(Bastion) / Private(Web) / Private(DB) 3개 서브넷

왜 나누는가

- 서브넷 = 보안 규칙의 단위. "Web 대역에서 DB 대역 2866만 허용"처럼 **대역 하나로 계층 전체를 지정** 가능
- 역할이 같은 서버끼리 묶여 있어 서버가 늘어도(Auto-Scaling) 규칙을 고칠 필요 없음
- Public Subnet만 1:1 공인 NAT IP를 가질 수 있음 → 공인 IP를 붙일 수 있는 위치 자체를 제한

나누지 않으면

- 모든 서버가 한 대역에 섞여 규칙을 서버 IP 단위로 관리 → 서버 증감마다 규칙 수정, 누락 발생
- 웹 서버 한 대가 침해되면 같은 대역의 DB까지 바로 접근 가능 (폭발 반경 확대)
- 실수로 DB 서버에 공인 IP를 붙여도 막아 줄 구조가 없음

IP 대역을 처음에 설계해야 하는 이유

- 서브넷 CIDR은 생성 후 변경이 어려움. 서버·LB·DB가 올라간 뒤에는 사실상 재구축
- 나중에 온프레미스(VPN·Direct Connect)나 다른 VPC(Peering·Transit Gateway)와 연결할 때 **대역이 겹치면 연결 불가** (일반 클라우드 공통 제약)
- 프로젝트당 VPC 5개 한도 → 시스템을 VPC로 나눌지 서브넷으로 나눌지 초기에 결정

3. 인터넷 출입구는 하나, Outbound도 통제

구성: VPC 경계의 Internet Gateway에 Firewall을 붙이고, 규칙은 아래만 허용

출발지	목적지	포트	방향	이유
0.0.0.0/0	LB 서비스 IP 대역(/27)	HTTP 80	Inbound	사용자 요청은 LB로만
내 PC 공인 IP	Bastion	RDP 3389	Inbound	관리 접속은 Bastion으로만
Public·Private Subnet	0.0.0.0/0	80·443	Outbound	패치·패키지 설치용

왜 출입구를 하나로 모으는가

- 통제·로그 지점이 하나 → 차단·감사가 한 곳에서 끝남
- 경계 Firewall은 Allow·Deny와 우선순위를 지원 → 특정 IP 일괄 차단 같은 대응을 서버 수와 무관하게 적용

왜 Outbound까지 막는가 (외부 지식 보강)

- 서버가 침해되면 공격자는 외부 서버로 데이터를 보내거나 명령을 받아 옴(C2 통신)
- Outbound를 80·443으로만 열어 두면 이런 통신 경로가 좁아지고, 이상 트래픽이 로그에 드러남
- 기본 Deny All 덕분에 "열지 않은 것은 막혀 있음"이 보장

> **규칙 작성 시 흔한 실수**
>
> - 내부 주소는 NAT IP가 아닌 내부 IP 대역으로 입력
> - 테스트가 안 된다고 Any IP·Any Port로 여는 것은 원인을 덮고 위험만 남김
> - 경로에 Firewall이 여러 개면(IGW·LB·Peering) 모두 허용해야 통신됨

4. Private 서버의 인터넷 - NAT Gateway

구성: Web 서버가 있는 Private Subnet에 NAT Gateway 연결. Web 서버 자체에는 공인 IP 없음

왜 필요한가

- Private 서버도 OS 패치, 패키지 설치(실습의 WordPress 구성), 외부 API 호출이 필요
- NAT Gateway는 **내부에서 시작한 Outbound만** 대표 공인 IP 1개로 변환 → 외부에서 먼저 들어오는 연결은 불가
- "인터넷은 쓰되 인터넷에서는 보이지 않는" 상태를 만드는 장치

대안과 비교

방법	결과
서버마다 1:1 공인 IP	서버 전체가 외부에서 접근 가능해짐. 공인 IP 관리·비용 증가
인터넷 차단	보안상 깔끔하지만 패치·라이브러리 설치 불가 → 취약점 방치
NAT Gateway	나가는 통신만 허용, 공인 IP 1개로 통일

운영상 이점 (외부 지식 보강)

- 외부 기관·SaaS가 접속 IP 화이트리스트를 요구할 때 **대표 IP 하나만 등록**하면 됨
- Auto-Scaling으로 서버가 바뀌어도 나가는 IP는 그대로

고려점

- NAT Gateway·Public IP는 과금 대상 → 정말 Outbound가 필요한 서브넷에만 연결
- NAT Gateway가 있어도 Firewall Outbound 규칙이 없으면 나가지 못함

5. Firewall과 Security Group을 둘 다 쓰는 이유

구분	Firewall	Security Group
위치	VPC 경계 (IGW·LB)	서버 vNIC(자원) 단위
보는 트래픽	North-South (외부 ↔ VPC)	North-South + East-West (VM ↔ VM, Subnet ↔ Subnet)
규칙	Allow·Deny, 우선순위 있음	Allow만, 순서 없음
대상 지정	출발지·목적지 IP	대상 주소 (Inbound는 출발지, Outbound는 목적지)

하나만 쓰면 생기는 문제

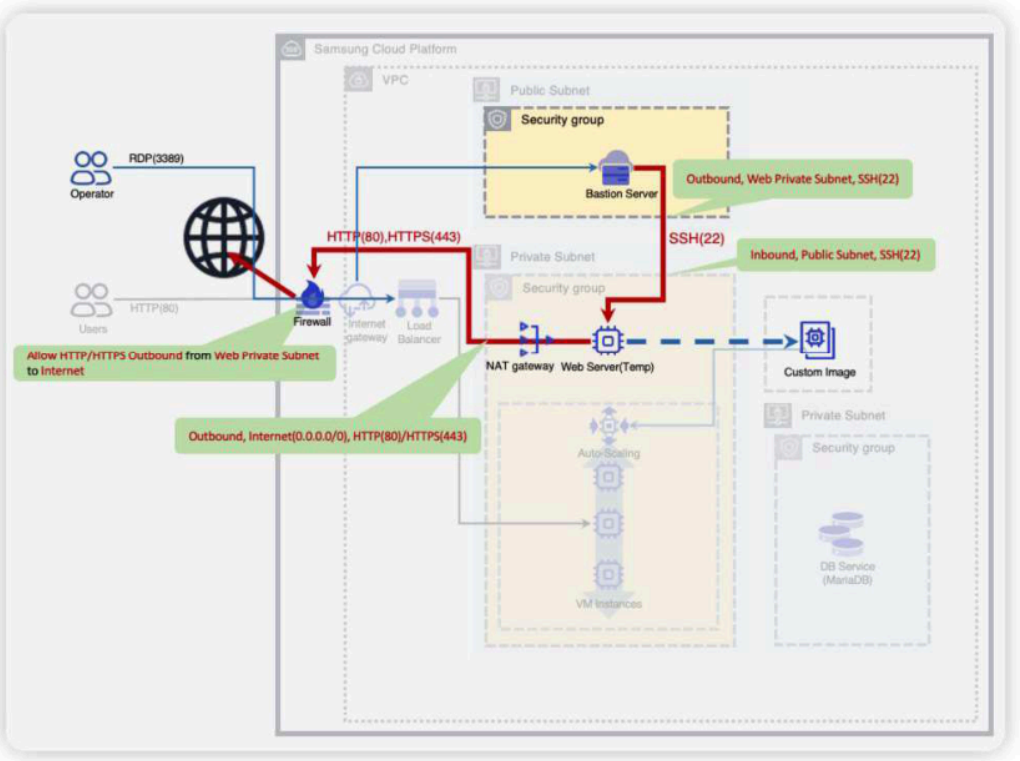
- **Firewall만**: VPC 안쪽 통신은 통제 불가. 웹 서버가 침해되면 같은 VPC의 DB·다른 서버로 자유롭게 이동(측면 이동, Lateral Movement)
- **Security Group만**: 경계에서 일괄 차단(Deny)과 우선순위 정책이 없음. 서버마다 규칙이 흩어져 전체 정책을 한눈에 보기 어려움

SG를 작게 나누는 이유 (실습의 To-Be 구조)

- Bastion SG / WebServer SG / DB SG처럼 **같은 규칙을 공유하는 역할 단위**로 생성
- 하나의 큰 SG에 모든 서버를 넣으면 DB에 필요한 규칙이 웹 서버에도 열림 → 최소 권한 위반
- 역할별 SG는 서버가 늘어도 해당 SG만 붙이면 끝

> **개념 - 다층 방어(Defense in Depth)**
>
> - 한 겹이 뚫리거나 잘못 설정되어도 다음 겹이 막도록 통제를 겹쳐 두는 설계
> - 이 실습에서는 경계(Firewall) → 위치(Private Subnet) → 자원(SG) → 비표준 포트(DB 2866) 순
> - 각 겹은 서로 다른 실수를 막음: Firewall은 외부 노출, SG는 내부 확산, Subnet은 공인 IP 부여 실수

6. Bastion Host - 관리 경로를 서비스 경로와 분리



구성과 규칙 흐름

- 내 PC 공인 IP → RDP 3389 → Bastion(Windows, Public Subnet)
- Bastion SG Outbound: Web Private Subnet SSH 22, DB Subnet 2866
- Web SG Inbound: Public Subnet CIDR에서 오는 SSH 22만 / DB SG Inbound: Bastion 대역의 2866
- Web·DB 서버에는 공인 IP가 없으므로 Bastion을 거치지 않으면 관리 접속 자체가 불가

왜 이렇게 하는가

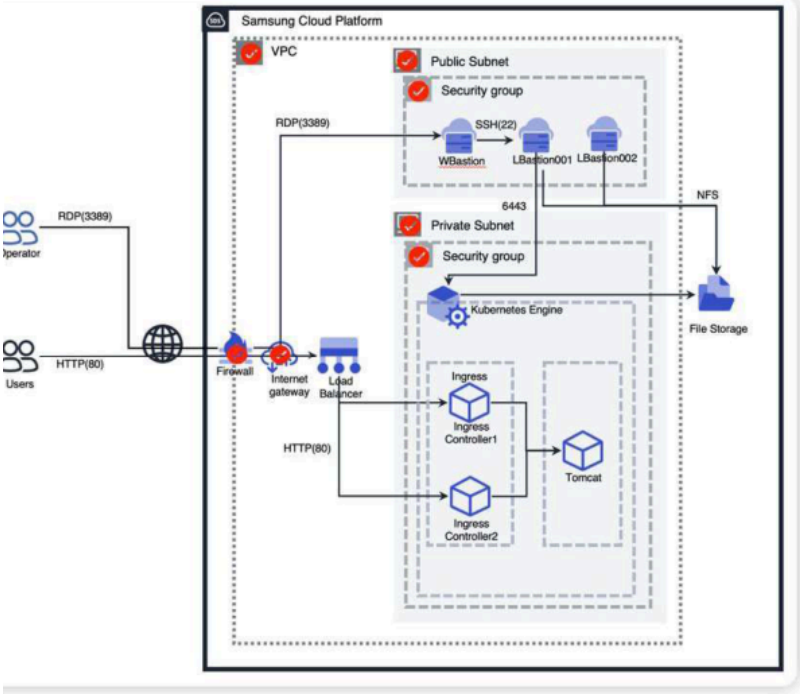
- 관리 포트를 인터넷에 여는 서버가 **Bastion 1대**로 줄어듦
- 출발지를 "내 PC 공인 IP"로 한정 → 허용된 장소에서만 관리 가능
- 접속 기록이 Bastion 한 곳에 모임 → 누가 언제 어떤 서버에 들어갔는지 추적

하지 않으면 (외부 지식 보강)

- 22·3389가 열린 공인 IP는 인터넷 스캐너에 수 분 안에 발견되고, 무차별 대입 공격(Brute Force)을 계속 받음
- 서버마다 관리 포트가 열려 있으면 패치가 늦은 서버 하나가 전체의 진입점이 됨

> **원격 접속이 안 될 때 (실습 Challenge 점검 순서)**
>
> - VPC에 Internet Gateway 연결 → 서버가 Public Subnet에 있는지 → NAT 활성화 여부
> - IGW Firewall에 내 PC IP → 서버 Private IP(또는 대역) RDP·SSH Inbound Allow (Deny보다 우선)
> - 서버 SG에 내 PC IP Inbound 규칙
> - 출발지를 0.0.0.0/0으로 여는 것은 금지. 내 PC 공인 IP를 특정

7. Bastion 운영 - Windows·Linux 2단 구성과 Bastion 자체 보호



Hands-on Lab II의 관리 동선 (Kubernetes 부분 제외)

- Operator → RDP 3389 → `WBastion` (Windows) → SSH 22 → `LBastion001/002` (Linux)
- `LBastion002`는 NFS로 File Storage를 마운트해 공유 데이터 관리

왜 Windows Bastion을 앞에 두는가

- DB Workbench, pgAdmin 같은 GUI 관리 도구를 Bastion 안에서 실행 → 관리자 PC에 DB 접속 경로를 열 필요 없음
- 관리자 PC에서는 RDP 한 경로만 허용하면 되고, 나머지 SSH·DB 접속은 모두 VPC 내부에서 발생

Bastion이 뚫리면 전체가 뚫림 → Bastion 자체 보호 (외부 지식 보강)

위험	대응
초기 비밀번호 유출	VM 생성 시 root 계정 정보가 **암호화되지 않은 메일**로 발송됨 → 받는 즉시 변경 (콘솔 경고 문구)
Key pair 분실·유출	Private Key는 최초 1회 다운로드. 공유 폴더·메신저 보관 금지
상시 노출	사용하지 않는 시간에는 중지하거나 규칙 비활성화
권한 남용	개인 계정 분리, 접속·명령 이력 기록, OS 패치 우선 적용

- Bastion에는 서비스용 프로그램·데이터를 두지 않음 → 침해 시 피해 범위와 공격 동기를 줄임

8. Load Balancer를 앞에 두는 이유

구성: Users → Firewall → IGW → LB 서비스(VIP:80) → 서버그룹(Auto-Scaling Web 서버)

LB가 해결하는 문제

문제	LB 없이	LB가 있을 때
서버 1대 장애	그 서버로 가는 요청 모두 실패	헬스체크로 제외하고 나머지로 분산
트래픽 급증	한 서버에 몰려 응답 지연	서버그룹 전체로 분산, Auto-Scaling과 연동
서버 증감	DNS·접속 주소를 매번 변경	서비스 IP(VIP) 고정, 뒤의 서버만 바뀜
노출	서버마다 공인 IP 필요	LB 서비스 IP만 공개, 서버는 Private

선택 기준

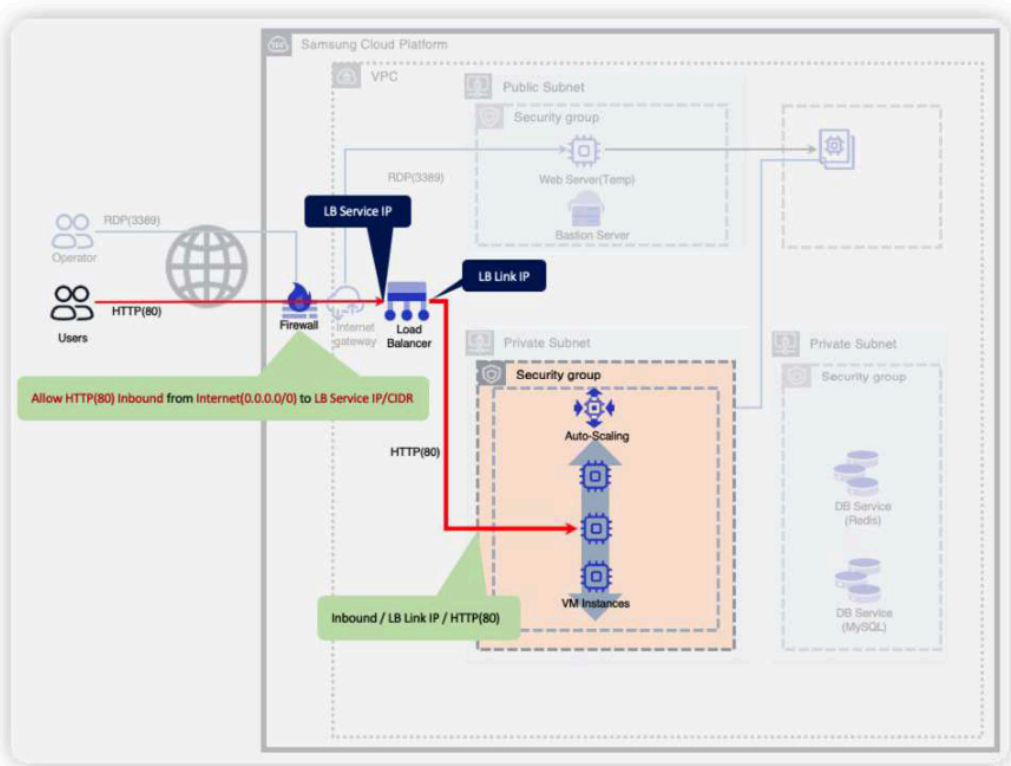
- **L4(TCP)**: 포트 단위 전달, 빠르고 단순 / **L7(HTTP·HTTPS)**: URL·헤더 기반 처리, 인증서 처리 (실습은 L7 HTTP)
- **Round Robin**: 서버 성능이 같을 때 / **Least Connection**: 요청 처리 시간이 들쭉날쭉할 때 / **IP Hash**: 같은 사용자를 같은 서버로 보내야 할 때(세션 유지)
- **크기**: Small은 NAT IP 20개, Medium은 100개까지. 서비스 수와 동시 세션 기준으로 선택하고 요금도 크기별로 다름

DNS 라운드로빈과 다른 점 (외부 지식 보강)

- DNS는 서버 상태를 모르고 캐시(TTL) 때문에 장애 서버로 계속 보냄
- LB는 헬스체크로 실시간 판단 → 장애 서버를 수 초 단위로 빼고, 복구되면 다시 넣음

> **개념 - 단일 장애 지점(SPOF)**
> - 그 하나가 멈추면 전체가 멈추는 구성 요소
> - 서버를 2대 이상 두어도 앞단 분배 장치가 없으면 사용자는 여전히 특정 서버 IP에 묶임
> - LB 자체는 클라우드가 이중화해 제공하므로 사용자는 서버 계층의 이중화에 집중

9. LB Link IP - 규칙을 정확히 쓰기 위한 개념



동작

- LB는 VPC 안의 서버와 통신할 때 자기 내부 IP인 **LB Link IP**를 출발지로 사용
- 서버 입장에서 요청을 보낸 상대는 사용자 IP가 아니라 LB Link IP

그래서 필요한 규칙 두 개

위치	규칙	의미
IGW Firewall	0.0.0.0/0 → LB 서비스 IP 대역, HTTP 80 Inbound	인터넷에서 LB까지
Web 서버 SG	LB Link IP → HTTP 80 Inbound	LB에서 서버까지

모르면 생기는 문제

- SG에 사용자 대역이나 LB 서비스 IP를 넣고 Link IP를 빠뜨림 → 헬스체크 실패 → 서버그룹 전체 Down → 서비스 중단
- 원인을 모른 채 SG를 0.0.0.0/0으로 열어 "해결" → Private 서버가 VPC 내부 전체에 열림

운영에서 이어지는 문제 (외부 지식 보강)

- 웹 서버 접근 로그에 사용자 IP 대신 Link IP만 남음
- L7 LB는 원래 사용자 IP를 `X-Forwarded-For` 헤더로 전달하는 것이 일반적 → 웹 서버 로그 형식에 이 헤더를 남기도록 설정해야 보안 분석·통계 가능

> **LB 서비스가 Down일 때 확인 순서**
>
> - 서버 SG에 LB Link IP Inbound 허용 여부
> - 헬스체크 포트에서 실제로 서비스가 기동 중인지
> - LB Firewall 사용 시 외부 → LB 서비스 IP 규칙, IGW Firewall 규칙

10. DB를 전용 Private Subnet에 두는 이유

구성: DB Service(MariaDB)를 DB 전용 Private Subnet에 생성. 포트는 기본값(3306) 대신 2866

출발지	목적지	허용	용도
Web Private Subnet (Auto-Scaling VM)	DB SG	TCP 2866	애플리케이션 쿼리
Bastion (Public Subnet)	DB SG	TCP 2866	관리자 Workbench 접속
그 외	DB SG	없음 (Deny All)	-

왜 전용 서브넷인가

- 데이터는 시스템에서 가장 가치가 높은 자산 → 가장 깊은 곳에, 가장 적은 경로로
- Web 계층이 침해되어도 공격자가 할 수 있는 것은 "2866으로 쿼리" 하나 → SSH 등으로 DB 서버에 직접 들어갈 수 없음
- DB 서브넷은 NAT·공인 IP가 필요 없음 → 외부와의 연결 자체가 없음

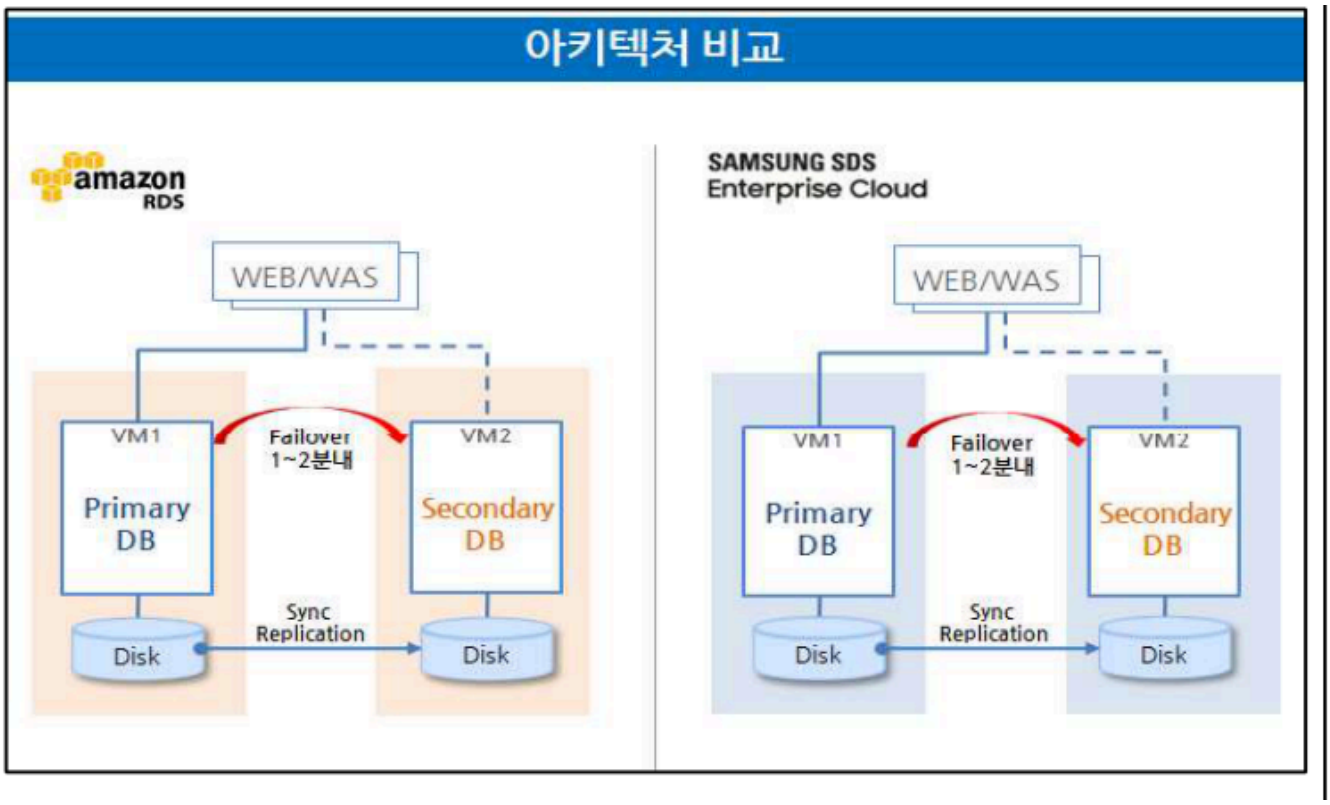
비표준 포트를 쓰는 이유와 한계 (외부 지식 보강)

- 인터넷 자동 스캐너는 3306·5432 같은 기본 포트부터 두드림 → 비표준 포트는 노출 시 발견 가능성을 낮춤
- 다만 포트 변경은 **보조 수단**일 뿐. 실제 보호는 서브넷 위치와 SG 출발지 제한이 담당

하지 않으면

- DB가 Web과 같은 대역에 있으면 Web 서버의 모든 포트 규칙이 DB에도 영향을 줌
- DB에 공인 IP·인터넷 경로가 생기면 데이터 유출 경로가 생김
- 실습 정리 문구: "Database는 반드시 Private Subnet에서 구성"

11. DB 고가용성 - Active-Standby



구성

- Primary DB(VM1)와 Secondary DB(VM2)를 서로 다른 Zone에 두고 디스크를 **동기 복제(Sync Replication)**
- Primary 장애 시 Standby로 자동 Fail-over, 1~2분 이내 (AWS RDS 등 타 CSP와 유사한 구조)
- SQL Server는 DB 자체의 AlwaysOn 기능으로 이중화

왜 동기 복제인가

- Primary에 기록된 데이터가 Standby에도 기록된 뒤에 완료 처리 → Fail-over 후에도 **커밋된 데이터 유실 없음**
- 대가: 쓰기마다 두 곳 기록을 기다리므로 단일 구성보다 쓰기 지연이 약간 늘어남

하지 않으면

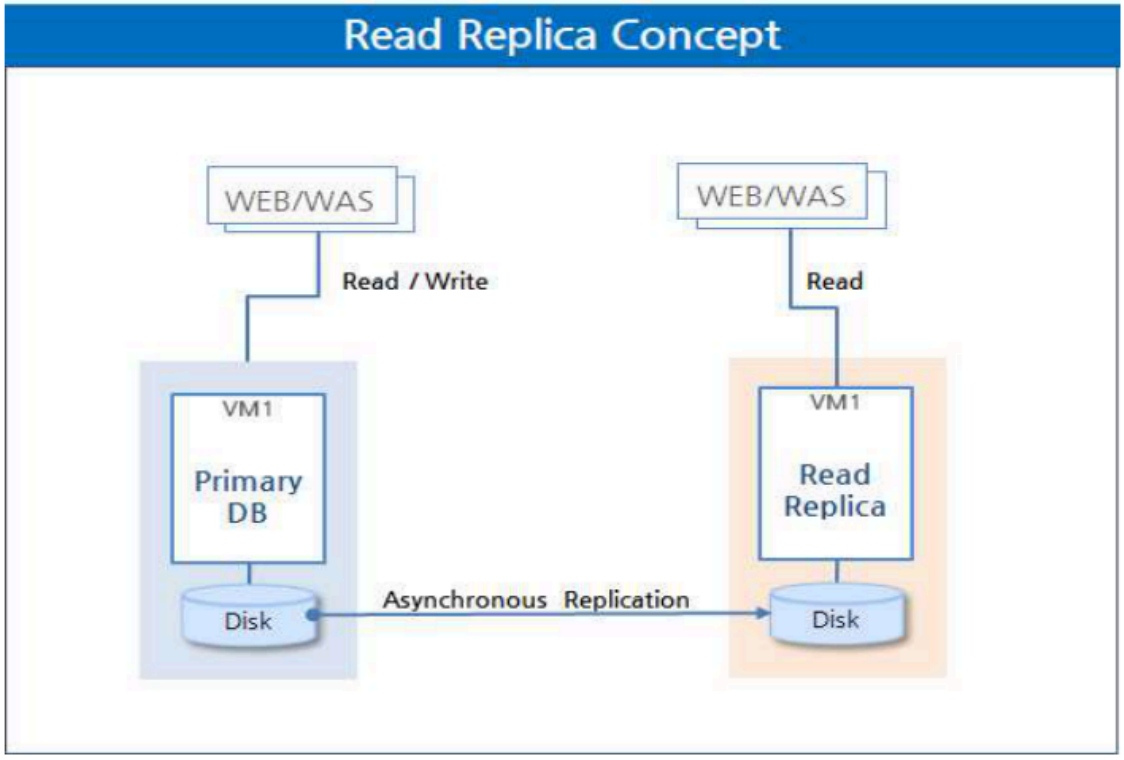
- DB 1대 = 시스템 전체의 SPOF. 웹 서버를 아무리 늘려도 DB가 멈추면 서비스 전체 중단
- VM·스토리지·Zone 장애 시 복구 방법은 백업에서 재구성뿐 → 복구 시간 수십 분~수 시간 + 마지막 백업 이후 데이터 유실

비용과 운영 고려

- 이중화 시 볼륨을 공유하지 않고 **DATA 블록 스토리지 2벌로 과금**
- Fail-over 순간 기존 DB 연결이 끊김 → 애플리케이션에 재접속(커넥션 풀 재연결) 로직이 있어야 1~2분 뒤 자동 회복 (외부 지식 보강)

> **개념 - RPO와 RTO**
>
> - RPO(Recovery Point Objective): 장애 시 잃어도 되는 데이터 기간. 동기 복제 HA는 사실상 0
> - RTO(Recovery Time Objective): 서비스가 다시 살아나기까지 허용 시간. HA는 1~2분, 백업 복구는 훨씬 김
> - 업무가 요구하는 RPO·RTO가 HA 필요 여부와 구성 방식을 결정

12. Read Replica - 읽기 분산과 DR



구성

- Primary(읽기·쓰기)의 데이터를 **비동기 복제(Async Replication)** 한 읽기 전용 DB
- Primary 1대당 최대 5개. PostgreSQL, MariaDB, EPAS, MySQL 지원
- 같은 VPC뿐 아니라 다른 VPC·원격지에도 생성 가능 (DR 목적이면 사전에 VPC Peering 필요)

HA와 무엇이 다른가

구분	Active-Standby (HA)	Read Replica
목적	가용성 (장애 대비)	성능 (읽기 확장), DR
복제 방식	동기	비동기
평상시 Standby 사용	불가 (대기만)	읽기 쿼리 처리
데이터 지연	없음	수 초 이상 늦을 수 있음

왜 필요한가

- BI·배치·통계 같은 무거운 조회가 Primary에서 돌면 주문·결제 같은 쓰기 업무가 느려짐
- 읽기를 Replica로 보내면 Primary는 쓰기에 집중

설계 시 주의 (외부 지식 보강)

- 비동기라 방금 쓴 데이터가 Replica에 아직 없을 수 있음 → "쓰고 바로 확인"하는 화면은 Primary에서 조회
- Replica는 HA 대체 수단이 아님. Primary 장애 시 자동 전환되지 않으므로 HA와 함께 구성
- 애플리케이션이 읽기·쓰기 연결을 분리해야 효과가 있음

13. 백업·복구 - HA로는 막지 못하는 사고

SCP DB 백업 구조

- Console에서 백업 주기·시간대 지정, 보관 기간 7~35일, 백업 파일은 Object Storage에 저장
- 복구는 백업 시점 또는 사용자 지정 시점 기준 (예: 현재 시각 1시간 전)
- 복구는 **신규 OS 기반 DB를 새로 만들고** 백업 파일을 Restore
- 복구 VM은 원본 생성 당시 OS·DB 버전, 가동 관리만 제공 (Scale-Up·백업·모니터링 미제공) → 필요한 데이터를 추출해 쓰는 용도

HA가 있는데 왜 백업이 따로 필요한가 (외부 지식 보강)

- HA는 데이터를 **그대로** 복제 → 실수로 `DELETE` · `DROP` 한 것, 랜섬웨어 암호화, 애플리케이션 버그로 망가진 데이터도 Standby에 즉시 복제
- 이런 논리적 장애를 되돌리는 방법은 "사고 이전 시점"으로 돌아가는 백업뿐

복구를 새 DB로 하는 이유

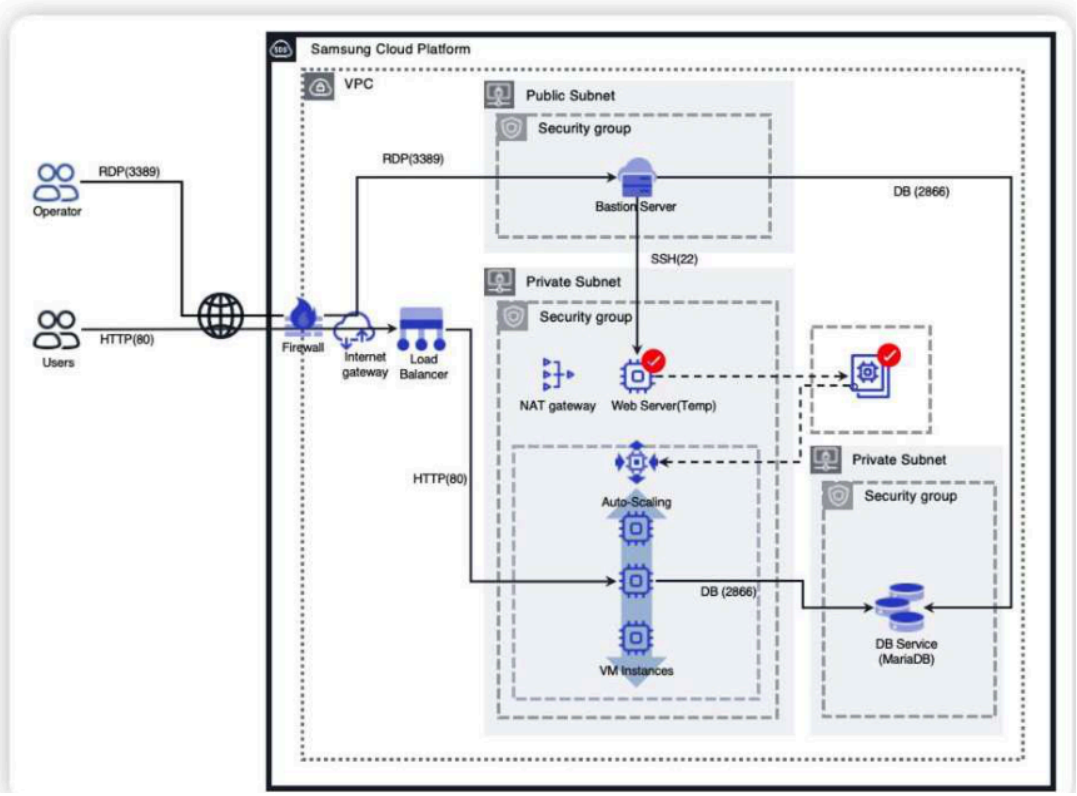
- 원본 위에 덮어쓰면 잘못된 시점을 고른 경우 되돌릴 방법이 없음 (2차 사고)
- 새 DB에서 데이터를 확인한 뒤 필요한 부분만 원본에 반영 → 안전한 복구 절차

운영 설정

- 파라미터: Dynamic은 온라인 반영, Static은 DB 재시작 필요 → 운영 중 변경은 작업 시간 계획
- 백업 시간대는 업무 부하가 낮은 시간으로

> **백업은 복구해 봐야 백업**
>
> - 보관 기간이 사고 발견까지 걸리는 시간보다 짧으면 쓸 백업이 없음
> - 정기적으로 복구 테스트를 해서 실제 복구 시간과 절차를 확인
> - 백업 버킷(Object Storage) 접근 권한도 최소화

14. Custom Image - 한 번 만들고 복제하는 서버



실습 흐름: Web Server(Temp)를 Private Subnet에 생성 → Bastion에서 SSH로 접속해 WordPress 구성(패키지는 NAT Gateway로 설치) → Custom Image(`webimg`) 생성 → Launch Configuration에 지정 → Auto-Scaling Group이 이 이미지로 서버 생성

왜 임시 서버를 거치는가

- 구성 작업은 한 번만, 이후 서버는 이미지에서 찍어 냄 → 모든 서버가 같은 상태로 시작
- Scale-Out으로 생긴 서버가 **즉시 서비스 가능한 상태**여야 LB 헬스체크를 통과하고 부하를 받음

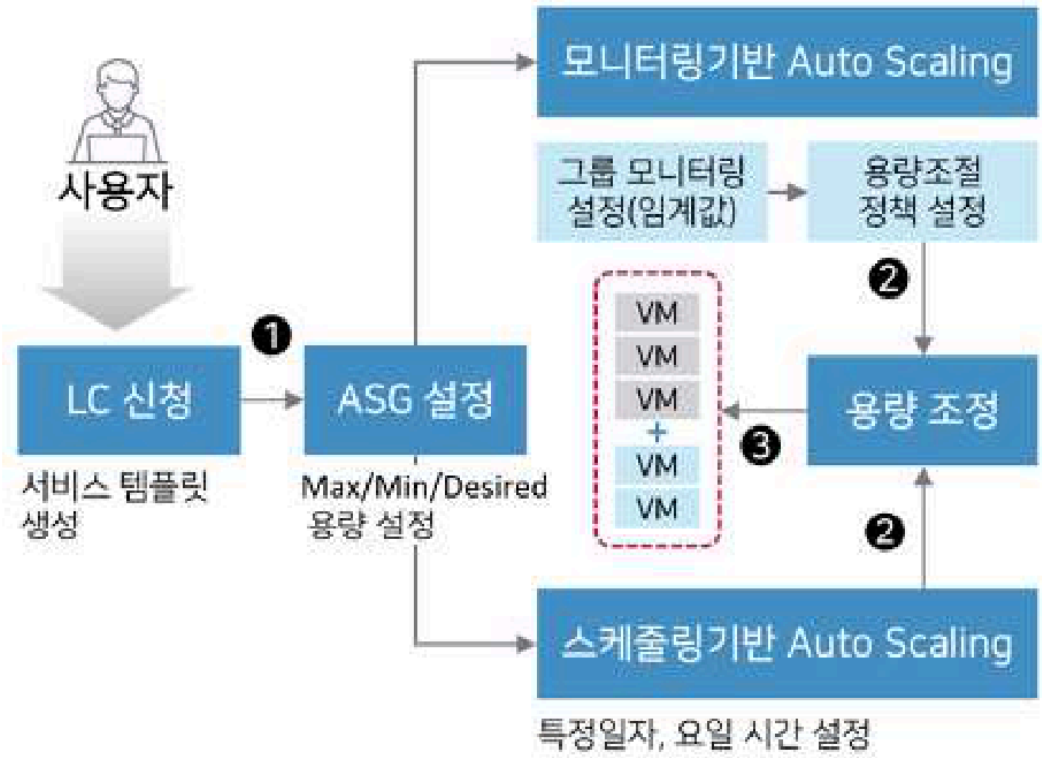
하지 않으면

- 기본 OS 이미지로 Scale-Out → 애플리케이션이 없는 빈 서버가 서버그룹에 붙어 오류 응답
- 서버를 손으로 하나씩 구성하면 패키지 버전·설정이 조금씩 달라짐(Configuration Drift) → "이 서버에서만 나는 장애"가 생김

이미지 운영 (외부 지식 보강)

항목	방법
패치·변경	운영 서버를 고치지 않고 이미지를 새로 만들어 Launch Configuration 교체
버전 관리	이미지 이름에 날짜·버전 표기, 직전 이미지는 롤백용으로 보관
비밀 정보	비밀번호·키를 이미지에 넣지 않음. 이미지를 쓰는 모든 서버에 복제됨
Init script와 조합	무거운 설치는 이미지에, 환경별 설정(접속 주소 등)은 Init script로

15. Auto-Scaling 구조 - 서버 수를 수요에 맞추기



구성 요소

- ① LC(Launch Configuration) 신청: 서비스 템플릿. OS 이미지(Custom Image), 서버 사양, 디스크
- ② ASG 설정: Min / Max / Desired 서버 수, VPC·Subnet·SG·LB 연결
- ③ 용량 조정: **모니터링 기반**(그룹 모니터링 임계값) 또는 **스케줄 기반**(특정 일자·요일·시간)

Min / Max를 정하는 의미

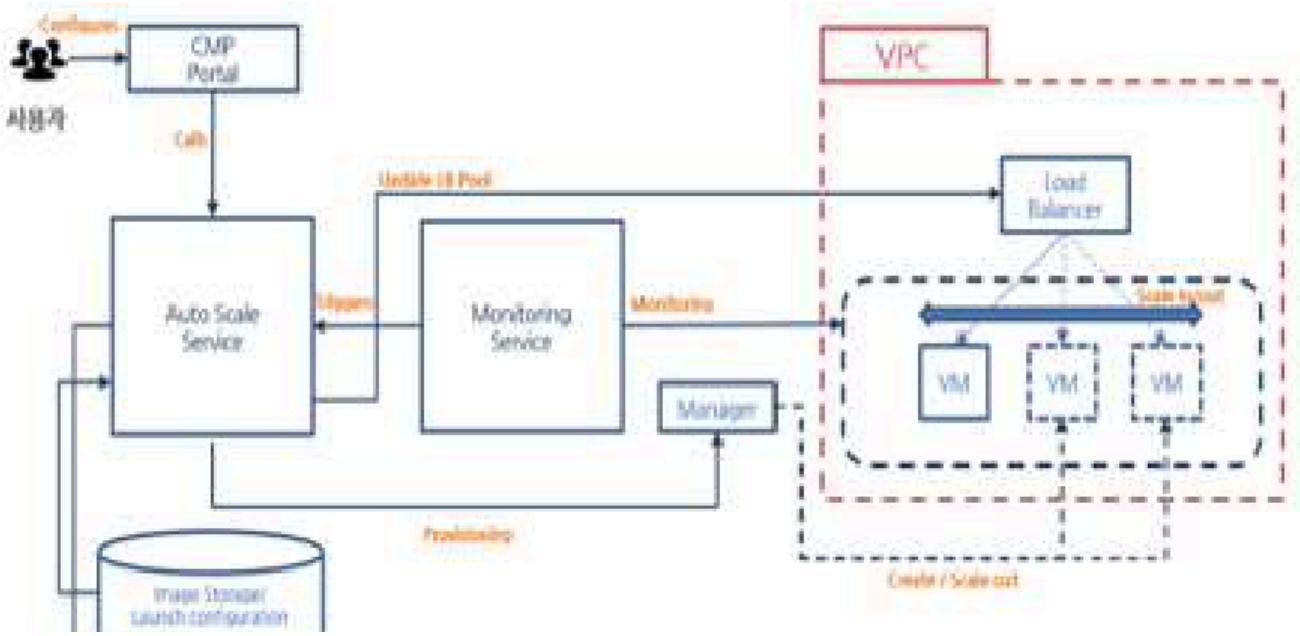
값	설계 의미	잘못 정하면
Min	평상시 최소 가용성. 2 이상이면 1대 장애에도 서비스 지속	1이면 그 서버 장애 시 새 서버가 뜰 때까지 중단
Max	비용 상한이자 DB·외부 연동이 견딜 수 있는 한계	너무 크면 공격성 트래픽에 비용 폭증, DB 커넥션 고갈
Desired	현재 목표 수. 수동 조정 가능	-

왜 필요한가

- 고정 대수 운영은 피크에 맞추면 평상시 낭비, 평상시에 맞추면 피크에 장애
- Auto-Scaling 서비스 자체는 과금되지 않고 **생성된 서버에만 과금** → 쓴 만큼만 비용

> **개념 - Stateless 설계**
>
> - Auto-Scaling 서버는 언제든 생기고 사라짐 → 서버 안에 세션·업로드 파일을 두면 Scale-In 때 유실
> - 세션은 Redis(CacheStore), 파일은 File/Object Storage, 데이터는 DB로 분리
> - 서버는 "버려도 되는 계산 자원"이 되어야 자동 확장이 안전함 (외부 지식 보강)

16. Scaling 정책 설계 - 임계치와 충돌



동작 흐름: Monitoring Service가 VM 지표 수집 → Auto Scale Service가 정책 판단 → 이미지 저장소의 Launch Configuration으로 Provisioning → LB Pool(서버그룹) 갱신

실습 정책과 실제 정책

구분	실습 (이벤트 강제 발생용)	실제 예시
Scale-Out	Memory ≥ 10%, 쿨다운 60초	CPU ≥ 90%
Scale-In	CPU ≤ 10%, 쿨다운 60초	CPU ≤ 10%

왜 Scale-Out 정책을 지우고 Scale-In을 시험했나

- 낮은 자원에서도 Out되도록 만든 정책과 In 정책이 동시에 참이 됨 → 늘었다 줄었다를 반복(Flapping)
- 실습이 보여 주는 교훈: **적절한 임계치로 무의미하거나 과도한 Scale-In/Out을 막는 것이 Auto-Scaling 설계의 본질**

임계치를 정하는 기준 (외부 지식 보강)

- 새 서버는 생성 → 부팅 → 헬스체크 통과까지 수 분이 걸림 → Out 임계치가 너무 높으면 서버가 뜨기 전에 기존 서버가 한계에 도달
- Out과 In 사이에 충분한 간격을 두고, 쿨다운으로 지표가 안정될 시간을 확보
- 출근 시간·이벤트처럼 **예측 가능한 피크는 스케줄 기반**으로 미리 늘려 두고, 예측 못 한 부하는 모니터링 기반으로 대응
- 실제 값은 안정화 기간 모니터링으로 도출 (튜토리얼 안내와 동일)

> **Scale-In 때 생기는 문제**
>
> - 줄어드는 서버에서 처리 중이던 요청·세션이 끊길 수 있음
> - 서버에 로그를 쌓아 두면 서버와 함께 사라짐 → 로그는 외부 저장소로 전송
> - 이벤트 알림을 설정해 Scale 동작을 사람이 인지할 수 있게

17. File Storage - 여러 서버가 같은 파일을 봐야 할 때

스토리지 3종의 역할

구분	Block Storage	File Storage	Object Storage
개념	서버에 직접 붙이는 고가용성 디스크	여러 클라이언트가 함께 쓰는 공유 저장소(NAS)	웹(HTTP) 인터페이스 기반 저비용 저장소
접근	서버 1대	여러 서버 동시	어디서나 API·URL
용도	OS, DB 데이터	웹 콘텐츠 공유, Web/WAS 이중화	백업, 로그, 정적 파일

왜 이중화된 웹 서버에 File Storage가 필요한가

- 사용자가 웹 서버 A에 파일을 올리고 다음 요청이 LB에 의해 서버 B로 가면 **파일이 없음**
- 공유 스토리지에 저장하면 어느 서버로 가도 같은 파일 → 서버 수와 무관한 일관성
- Scale-In으로 서버가 사라져도 파일은 남음

프로토콜 선택

- NFS: Linux 간 공유 (Hands-on Lab II의 `LBastion002`가 NFS로 마운트)
- CIFS: Windows 환경, 윈도우-리눅스 간 공유 (SMB 확장)

접근 제어

- File Storage는 생성 후 **적용 서버**를 지정해야 그 서버만 접근 가능 → 스토리지 차원의 ACL
- 적용 서버를 넓게 잡으면 관계없는 서버도 공유 데이터를 읽고 지울 수 있음 → 필요한 서버만 등록

18. 장애 시나리오로 다시 보는 설계

구성도의 각 요소가 실제로 어떤 사고를 막는지 시나리오별 정리

장애·사고	동작하는 설계 요소	결과	그 요소가 없다면
웹 서버 1대 다운	LB 헬스체크 + ASG Min 유지	장애 서버 제외, 새 서버 자동 생성	일부 사용자 오류, 수동 복구까지 지속
트래픽 급증	모니터링 기반 Scale-Out + Custom Image	같은 구성의 서버 추가, LB 자동 등록	응답 지연·타임아웃
DB Primary 장애	Active-Standby Fail-over	1~2분 내 Standby로 전환, 데이터 유실 없음	백업 복구까지 장시간 중단 + 데이터 유실
실수로 테이블 삭제	백업 + 시점 복구(새 DB)	사고 이전 데이터 추출	복구 불가 (HA는 삭제도 복제)
관리 포트 스캔·대입 공격	Bastion 1곳 + 출발지 IP 제한	공격 표면 최소	모든 서버가 공격 대상
웹 서버 침해	DB 전용 서브넷 + SG(2866만)	공격자의 이동 경로 제한	같은 대역 DB·서버로 확산
침해 서버의 외부 통신	Firewall Outbound 제한	유출 경로 축소, 로그로 탐지	데이터 반출·원격 제어
Zone 장애	Multi-Zone 네트워크·DB 이중화	다른 Zone에서 서비스 지속	전체 중단

- 설계를 검토할 때는 "이 요소를 빼면 어떤 행이 깨지는가"로 필요성을 판단
- 반대로 어떤 장애 행에도 대응 요소가 없다면 그것이 설계의 빈틈

19. 설계 점검 체크리스트

영역	점검 질문
네트워크	서브넷이 역할(Public·Web·DB)별로 나뉘어 있는가
네트워크	IP 대역이 온프레미스·다른 VPC와 겹치지 않는가
노출	인터넷에서 닿는 자원이 LB와 Bastion뿐인가
노출	공인 IP가 붙은 서버 목록을 설명할 수 있는가
접근 제어	Firewall Inbound에 0.0.0.0/0 → 관리 포트 규칙이 없는가
접근 제어	SG가 역할 단위로 나뉘고, DB SG의 출발지가 Web·Bastion뿐인가
접근 제어	Outbound가 필요한 포트로만 제한되어 있는가
관리 경로	관리 접속이 Bastion을 거치고, Bastion 출발지가 지정 IP뿐인가
관리 경로	초기 비밀번호 변경, Key pair 보관이 되어 있는가
가용성	서버가 2대 이상이고 LB 헬스체크가 실제 서비스 포트를 보는가
가용성	DB가 HA 구성이고 애플리케이션이 재접속을 처리하는가
확장	Scale-Out·In 임계치 사이에 간격과 쿨다운이 있는가
확장	서버에 세션·파일·로그가 남지 않는가
표준화	서버가 Custom Image에서 생성되고 이미지 버전이 관리되는가
데이터	백업 보관 기간이 사고 발견 시간보다 길고 복구 테스트를 했는가
데이터	File Storage 적용 서버가 필요한 서버로 한정되는가
비용	ASG Max, LB 크기, NAT Gateway 위치가 필요 수준에 맞는가

20. 정리 - 이번 실습에서 얻은 설계 관점

1) 노출은 최소로, 입구는 목적별로 하나씩

- 사용자는 LB, 운영자는 Bastion. 나머지는 모두 Private
- 입구가 적을수록 지킬 곳과 볼 로그가 줄어듦

2) 한 겹이 아니라 여러 겹

- Firewall·Subnet·SG·포트가 서로 다른 실수를 막음
- 규칙 하나를 잘못 열어도 다음 겹이 버티도록 설계

3) 장애는 예외가 아니라 전제

- LB 헬스체크, ASG Min, DB Active-Standby는 모두 "언제든 한 대는 멈춘다"는 가정에서 출발
- HA와 백업은 막는 사고가 다름: HA는 물리 장애, 백업은 논리 장애

4) 서버는 교체 가능하게, 상태는 밖으로

- Custom Image로 같은 서버를 찍어 내고, 세션·파일·데이터는 Redis·File Storage·DB로
- 그래야 Auto-Scaling과 장애 복구가 사람 손 없이 동작

5) 자동화에도 판단이 필요

- 임계치·Min/Max·보관 기간은 콘솔이 정해 주지 않음
- 실습의 강제 임계치와 정책 충돌은 잘못된 값이 어떤 문제를 만드는지 보여 주는 사례

> **한 줄 요약**
>
> 구성도의 모든 박스와 화살표에는 막으려는 사고가 하나씩 있음. 설계를 볼 때 "이걸 빼면 무엇이 깨지는가"를 물으면 왜 그렇게 구성했는지가 보임